

Velox: Scalable Fair Asynchronous MPC from Lightweight Cryptography

Akhil Bandarupalli
Purdue University
West Lafayette, IN, USA
abandaru@purdue.edu

Xiaoyu Ji
Tsinghua University
Beijing, China
jixy23@mails.tsinghua.edu.cn

Aniket Kate
Purdue University & Supra Research
West Lafayette, IN, USA
aniket@purdue.edu

Chen-Da Liu-Zhang
Lucerne University of Applied
Sciences and Arts & Web3 Foundation
Lucerne, Switzerland
chen-da.liuzhang@hslu.ch

Daniel Pöllmann
ETH Zurich & Lucerne University of
Applied Sciences and Arts
Lucerne, Switzerland
dpollmann@ethz.ch

Yifan Song
Tsinghua University & Shanghai Qi
Zhi Institute
Beijing, China
yfsong@mail.tsinghua.edu.cn

A ARTIFACT APPENDIX

We discuss the artifact of our work Velox. We divide this section into three components: (a) Availability, (b) Functionality/Reusability, and (c) Reproducibility.

A.1 Availability

Our artifact contains the implementation of our described MPC protocol. Further, it also demonstrates the application of our protocol to Anonymous Broadcast application. We implemented our protocol in the Rust programming language. The artifact is available on Zenodo and Github. On Zenodo, it is available at:

<https://doi.org/10.5281/zenodo.17018403>

On Github, it is available at:

<https://github.com/akhilb/Velox-MPC>

The results in the paper come from the commit ID:

75f6f6ab01c60ad0f9e23d182183e3127076b1a8

Hardware and Software Dependencies. Velox does not have any hardware dependencies. However, we ran the benchmarks in Amazon Web Services (AWS) using c5.large instances, each with 2 CPU cores and 4 GB of RAM, running an Ubuntu 24.04 Focal operating system. For storage, we add 100 GB of storage on each machine for log storage. Our artifact requires the Rust programming toolset to be installed. The artifact is stable at Rust version 1.83.0. Relevant installation instructions have been provided in the artifact’s README file.

A.2 Functionality and Reusability

Our README.md file in the artifact describes the sequential steps necessary to build and execute the artifact locally. Our artifact implements the described MPC protocol Π_{MainFair} by first implementing its the composing functionalities as modular building blocks, and then putting these blocks together. We describe the overview of these building blocks and their composition as a graph in fig. 1.

The graph can be interpreted as a Directed Acyclic Graph (DAG) of individual components. The source vertices of the DAG are the fundamental building blocks implemented as standalone components. These blocks work with an asynchronous runtime. The par-

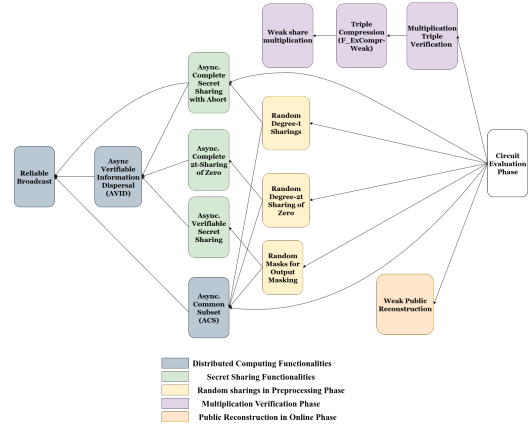


Figure 1: Functionality composition diagram of Velox.

ent protocol (the MPC protocol) treats these blocks as child processes. It passes inputs to these blocks and receive outputs from them through queues. For instance, when a party running Velox wants to use RBC to broadcast an input, the party sends to a *receiver* queue of the Reliable Broadcast block. This block listens to this queue, executes the protocol, and returns the output to a *sender* queue. The parent process listens to this queue, consumes the output, and executes further steps. Our architecture uses the tokio asynchronous runtime to implement and invoke these building blocks in an asynchronous and multi-threaded environment.

Our modular architecture enables reusability of our artifact. Future works relying on similar functionalities can directly employ our building blocks. A implemented the following protocols:

- (1) Cachin-Tessaro’s Reliable Broadcast protocol [CT05]
- (2) Dispersed Ledger’s Asynchronous Verifiable Information Dispersal protocol [YP⁺22]
- (3) Das *et al.*’s [DDL⁺24] Asynchronous Common Subset protocol
- (4) Our Asynchronous Complete Secret Sharing with Abort protocol
- (5) Our $2t$ -sharing with Abort protocol

A.3 Programming Model

In asynchronous distributed systems with complex workflows like Velox, liveness bugs can arise because of improper workflow orchestration. Unlike synchronous protocols where parties are expected to synchronize at the end of each stage, asynchronous protocols proceed at network speed, which can lead to different parties being at different stages in the protocol. This leads to situations in practice where parties can process messages out of order. For instance, a party P_j can process messages from depth $d_j + 10$ before processing messages from depth d_j . Hence, the implementation must take such scenarios into account and still be able to make progress.

Our programming model provides a seamless and easy to understand structure to implement such protocols, handle these out of order deliveries, and make progress by default. We decompose our entire protocol into sequential stages. In our model, each stage contains three types of functions or methods based on their prefix: (a) `init_`, (b) `handle_`, and (c) `verify_` functions. Functions with `init_` prefix are responsible for initializing the next stage and sending out messages to parties. The `handle_` functions are invoked by the networking module to handle messages received from other parties. Finally, the `verify_` functions check whether the current stage can be terminated and the next stage can be started.

We also have additional constraints for each type of function. The `init_` functions have at most once semantics or are idempotent. This can be achieved by either making sure that the function is invoked at most once or by using a counter variable corresponding to each stage of the protocol. So, the `init_` function executes the designated code only when the counter variable matches the number of the stage.

Next, the `handle_` functions only invoke the `verify_` functions. The `handle_` functions receive messages from parties and store them in appropriate data structures. These functions always invoke the `verify_` functions to invoke the verification procedure involved in verifying whether the stage has completed. For instance, in a reliable broadcast protocol, in the echo stage, the `handle_echo` function records the echo message and invokes `verify_echo_termination` function, which verifies if the party received n -t echos.

Finally, the `verify_` functions handle the core logic of the stage of the protocol. Each `verify_` function checks the required termination conditions for the protocol’s current stage. If the conditions succeed, the function invokes the `init_` function of the next stage of the protocol and the protocol moves to the subsequent stage. Note that the `verify_` function waits until the current stage of the protocol matches the stage designated by the specific function. Additionally, the `verify_` function of the current stage also invokes the `verify_` function of the next stage. In this way, a lagging party in an older stage can directly catch up to the current stage by sequentially verifying the termination conditions necessary for each stage.

A.4 Reproducibility

The results we reported in this paper have been performed in a Local Area Network (LAN) testbed on Amazon Web Services (AWS). We instantiated each party on an individual `c5.large` machine. We provided the scripts necessary for setting up the AWS environment in the artifact. These scripts have been derived from Narwhal’s

benchmarking code [DKKSS22]. For reproducing the results in the paper, first set up the environment on AWS by following the instructions in the README.md file attached with the artifact.

To reproduce the results in the paper, run Velox in a LAN testbed (with a single region for instance `us-east-1`) with $n = 16, 64, 112$ parties. In the $n = 16$ testbed, run Velox with the following configurations in the `fabfile.py` file.

- (1) `num_messages = 256, batch_size = 2000, compression_factor = 10`
- (2) `num_messages = 512, batch_size = 2000, compression_factor = 10`
- (3) `num_messages = 1024, batch_size = 2000, compression_factor = 10`

In the $n = 64$ testbed, run Velox with the following configurations in the `fabfile.py` file.

- (1) `num_messages = 256, batch_size = 500, compression_factor = 10`
- (2) `num_messages = 512, batch_size = 500, compression_factor = 10`
- (3) `num_messages = 1024, batch_size = 500, compression_factor = 10`

In the $n = 112$ testbed, run Velox with the following configurations in the `fabfile.py` file.

- (1) `num_messages = 256, batch_size = 300, compression_factor = 10`
- (2) `num_messages = 512, batch_size = 300, compression_factor = 10`

Reproducing results in Figure 1. For reproducing results of Figure 1 in the paper (phase-wise runtime), use the results of the instances with $n = 64$ parties.

Reproducing results in Figure 2. For reproducing results in Figure 2 in the paper (Anonymity set size k vs runtime), use the previous figure’s result for $n = 64$ and add the results of $n = 16$ parties into the paper.

Reproducing results in Figure 3. For reproducing results in Figure 3 in the paper (Scalability plot:), add the results with $n = 112$ parties to the figure.

In case evaluation on AWS is not possible, the trend in the results can be reproduced on a single powerful machine by following the instructions for running the code locally. Please make sure to provision one core per party.

A.5 Version

Based on the LaTeX template for Artifact Evaluation V20220926.

REFERENCES

- [CT05] C. Cachin and S. Tessaro. Asynchronous verifiable information dispersal. In *SRDS’05*, pages 191–201, 2005.
- [DDL⁺24] Sourav Das, Sisi Duan, Shengqi Liu, Atsuki Momose, Ling Ren, and Victor Shoup. Asynchronous consensus without trusted setup or public-key cryptography. *Cryptology ePrint Archive*, 2024.
- [DKKSS22] George Danezis, Leferis Kokoris-Kogias, Alberto Sonnino, and Alexander Spiegelman. Narwhal and tusk: a dag-based mempool and efficient bft consensus. In *EuroSys’22*, 2022.
- [YPA⁺22] Lei Yang, Seo Jin Park, Mohammad Alizadeh, Sreeram Kannan, and David Tse. {DispersedLedger}: {High-Throughput} byzantine consensus on variable bandwidth networks. In *USENIX NSDI’22*, pages 493–512, 2022.